



CHAPTER 02

Operators in Python

Lists • Tuples • Dictionaries

In the previous chapter, we learned about basic data types in Python. In this chapter, we will explore three of the most important and widely-used data structures in Python: **Lists**, **Tuples**, and **Dictionaries**. Each of these structures allows you to store and organize multiple pieces of data together. We will start from the very basics and gradually move to more advanced operations.

Learning Objectives

Understand what Lists, Tuples, and Dictionaries are and why we use them.
Perform basic and advanced operations on each of these data structures.
Learn how Python handles these structures internally with practical examples.

CLOUD & AI ACADEMY

1 Basic Operations on List

1.1 What is a List?

A List in Python is a collection that can hold multiple items in a single variable. Think of it like a row of storage boxes numbered from 0, where each box can hold any type of item — a number, a word, or even another list.

In Python, a list is written using `[ ]` (square brackets), and items inside it are separated by commas.

Syntax

SYNTAX

```
list_name = [item1, item2, item3, ...]
```

Example 1.1 — Creating a List

Let us create a list that stores the names of five programming languages:

PYTHON CODE

```
languages = ['Python', 'Java', 'C++', 'JavaScript', 'Ruby']
```

```
print(languages)
```

OUTPUT

```
['Python', 'Java', 'C++', 'JavaScript', 'Ruby']
```

Example 1.2 — Mixed Data in a List

A list can hold different types of values at the same time:

PYTHON CODE

```
student = ['Alice', 21, 'Computer Science', 9.1]
print(student)
```

OUTPUT

```
['Alice', 21, 'Computer Science', 9.1]
```

Remember

Lists in Python are ordered — they maintain the order in which items are added.

Lists are mutable — you can change, add, or remove items after the list is created.

Lists allow duplicate values — the same item can appear more than once.

1.2 Accessing List Elements

Each item in a list has an **index** — a number that represents its position. Python starts counting from **0**, not 1. This is called **zero-based indexing**.

PYTHON CODE

```
languages = ['Python', 'Java', 'C++', 'JavaScript', 'Ruby']

print(languages[0]) # First element
print(languages[2]) # Third element
print(languages[-1]) # Last element (negative indexing)
```

OUTPUT

```
Python
C++
Ruby
```

Negative Indexing

Python also allows negative indexes. -1 refers to the last item, -2 refers to the second last, and so on.

This is useful when you want to access items from the end of the list without knowing the total length.

1.3 Slicing a List

Slicing allows you to extract a **portion** of a list. The syntax is: `list[start:end]`. The slice includes items from index start up to — but **not including** — the item at index end.

PYTHON CODE

```
languages = ['Python', 'Java', 'C++', 'JavaScript', 'Ruby']

print(languages[1:4]) # Items at index 1, 2, 3
print(languages[:3]) # First 3 items
print(languages[2:]) # From index 2 to the end
```

OUTPUT

```
['Java', 'C++', 'JavaScript']
['Python', 'Java', 'C++']
['C++', 'JavaScript', 'Ruby']
```

1.4 Modifying a List

Since lists are mutable, you can update any element directly using its index.

PYTHON CODE

```
languages = ['Python', 'Java', 'C++', 'JavaScript', 'Ruby']

languages[1] = 'Kotlin' # Replacing 'Java' with 'Kotlin'
print(languages)
```

OUTPUT

```
['Python', 'Kotlin', 'C++', 'JavaScript', 'Ruby']
```

1.5 Adding Elements to a List

Python provides two main ways to add elements to a list:

append() — Add to the End

PYTHON CODE

```
languages = ['Python', 'Java', 'C++']

languages.append('Go') # Adds 'Go' at the end
print(languages)
```

OUTPUT

```
['Python', 'Java', 'C++', 'Go']
```

insert() — Add at a Specific Position

The `insert(index, value)` method adds an element at a specific position.

PYTHON CODE

```
languages = ['Python', 'Java', 'C++']

languages.insert(1, 'Swift') # Inserts 'Swift' at index 1
print(languages)
```

OUTPUT

```
['Python', 'Swift', 'Java', 'C++']
```

1.6 Removing Elements from a List

`remove()` — Remove by Value

PYTHON CODE

```
languages = ['Python', 'Java', 'C++', 'JavaScript']

languages.remove('Java') # Removes 'Java' from the list
print(languages)
```

OUTPUT

```
['Python', 'C++', 'JavaScript']
```

`pop()` — Remove by Index

The `pop()` method removes the item at a given index. If no index is given, it removes the last item.

PYTHON CODE

```
languages = ['Python', 'Java', 'C++', 'JavaScript']

removed = languages.pop(2) # Removes item at index 2
print('Removed:', removed)
print('List:', languages)
```

OUTPUT

```
Removed: C++
List: ['Python', 'Java', 'JavaScript']
```

1.7 List Length and Membership

Finding the Length — `len()`

PYTHON CODE

```
languages = ['Python', 'Java', 'C++', 'JavaScript', 'Ruby']
print(len(languages))
```

OUTPUT

5

Checking if an Item Exists — in

PYTHON CODE

```
languages = ['Python', 'Java', 'C++', 'JavaScript', 'Ruby']

print('Python' in languages)
print('Swift' in languages)
```

OUTPUT

```
True
False
```

1.8 Common List Methods — Quick Reference

Method	Description	Example
append(x)	Adds item x at the end of the list	lst.append(10)
insert(i, x)	Inserts item x at position i	lst.insert(0, 5)
remove(x)	Removes the first occurrence of x	lst.remove('Java')
pop(i)	Removes and returns item at index i	lst.pop(2)
sort()	Sorts the list in ascending order	lst.sort()
reverse()	Reverses the order of the list	lst.reverse()
index(x)	Returns the index of item x	lst.index('C++')
count(x)	Returns how many times x appears	lst.count(5)
clear()	Removes all items from the list	lst.clear()
copy()	Returns a shallow copy of the list	lst2 = lst.copy()

2 Complex List Handling

2.1 Iterating Through a List

You can use a for loop to go through each element in a list one by one. This is one of the most common operations you will perform.

PYTHON CODE

```
fruits = ['Mango', 'Apple', 'Banana', 'Orange']

for fruit in fruits:
    print(fruit)
```

OUTPUT

```
Mango
Apple
Banana
Orange
```

Iterating with Index using enumerate()

When you also need the index of each item, use `enumerate()`:

PYTHON CODE

```
fruits = ['Mango', 'Apple', 'Banana', 'Orange']

for index, fruit in enumerate(fruits):
    print(index, '-', fruit)
```

OUTPUT

```
0 - Mango
1 - Apple
2 - Banana
3 - Orange
```

2.2 List Comprehension

List Comprehension is a concise way to create a new list by applying an expression to each element of an existing list. It replaces multi-line for loops with a single line.

Standard Loop vs. List Comprehension

Suppose we want to create a list of squares of numbers from 1 to 5.

Using a standard loop:

PYTHON CODE — STANDARD LOOP

```
squares = []
for n in range(1, 6):
    squares.append(n * n)
print(squares)
```

Using list comprehension:

PYTHON CODE — LIST COMPREHENSION

```
squares = [n * n for n in range(1, 6)]
print(squares)
```

OUTPUT

```
[1, 4, 9, 16, 25]
```

List Comprehension with a Condition

Filter only even numbers from a list:

PYTHON CODE

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

evens = [n for n in numbers if n % 2 == 0]
print(evens)
```

OUTPUT

```
[2, 4, 6, 8, 10]
```

2.3 Nested Lists (2D Lists)

A Nested List is a list that contains other lists as its elements. This is often used to represent a table or a grid — similar to rows and columns in a spreadsheet.

PYTHON CODE

```
# A 3x3 matrix (3 rows, 3 columns)
matrix = [
```

```
[1, 2, 3],
[4, 5, 6],
[7, 8, 9]
]

print(matrix[0])    # First row
print(matrix[1][2]) # Row 1, Column 2 (value = 6)
```

OUTPUT

```
[1, 2, 3]
6
```

Iterating Through a Nested List**PYTHON CODE**

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

for row in matrix:
    for value in row:
        print(value, end=' ')
    print() # New line after each row
```

OUTPUT

```
1 2 3
4 5 6
7 8 9
```

2.4 Sorting a List

sort() — Modifies the Original List**PYTHON CODE**

```
marks = [78, 45, 90, 62, 55]

marks.sort()    # Ascending order
print('Ascending:', marks)

marks.sort(reverse=True) # Descending order
print('Descending:', marks)
```

OUTPUT

```
Ascending: [45, 55, 62, 78, 90]
Descending: [90, 78, 62, 55, 45]
```


sorted() — Returns a New Sorted List

Unlike `sort()`, the `sorted()` function does not change the original list. It creates and returns a new sorted list.

PYTHON CODE

```
marks = [78, 45, 90, 62, 55]

sorted_marks = sorted(marks)
print('Original:', marks)
print('Sorted:', sorted_marks)
```

OUTPUT

```
Original: [78, 45, 90, 62, 55]
Sorted: [45, 55, 62, 78, 90]
```

2.5 Combining and Copying Lists

Concatenating Lists with +

PYTHON CODE

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]

combined = list1 + list2
print(combined)
```

OUTPUT

```
[1, 2, 3, 4, 5, 6]
```

Extending a List with extend()

The `extend()` method adds all elements of one list to another, directly modifying the original.

PYTHON CODE

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]

list1.extend(list2)
print(list1)
```

OUTPUT

```
[1, 2, 3, 4, 5, 6]
```

Copying a List

Use `copy()` or `list[:]` to make a copy. Directly assigning (`list2 = list1`) does not create a new list — both variables point to the same list.

PYTHON CODE

```
original = [10, 20, 30]

copy1 = original.copy()
copy2 = original[:]

copy1.append(40)
print('Original:', original) # Not changed
print('Copy1:', copy1)
```

OUTPUT

```
Original: [10, 20, 30]
Copy1: [10, 20, 30, 40]
```

2.6 Working with the Python Shell — Lists

The following screenshot shows how list operations look when executed in the Python interactive shell (IDLE or terminal). Practice these operations in your own environment.

[SCREENSHOT]

Screenshot: Python interactive shell showing list creation, indexing, append, remove, and sort operations

3 Basic Operations on Tuple

3.1 What is a Tuple?

A Tuple is very similar to a List — it also stores multiple items in a single variable. However, there is one very important difference:

Key Difference: Tuple vs. List

A List is mutable — you can change its contents after creation.

A Tuple is immutable — once created, its contents cannot be changed.

Tuples are used when you want to store data that should not be modified, such as coordinates, database records, or configuration settings.

Tuples are written using `()` (parentheses) and items are separated by commas.

Syntax

SYNTAX

```
tuple_name = (item1, item2, item3, ...)
```

Example 3.1 — Creating a Tuple

PYTHON CODE

```
coordinates = (28.7041, 77.1025) # Latitude, Longitude of Delhi
print(coordinates)
print(type(coordinates))
```

OUTPUT

```
(28.7041, 77.1025)
<class 'tuple'>
```

3.2 Accessing Tuple Elements

Just like lists, tuples use zero-based indexing to access elements:

PYTHON CODE

```
student = ('Alice', 21, 'Computer Science', 9.1)

print(student[0]) # First element
print(student[-1]) # Last element
```

OUTPUT

```
Alice
```

9.1

3.3 Tuple Slicing

Slicing works exactly the same as in lists:

PYTHON CODE

```
colors = ('Red', 'Green', 'Blue', 'Yellow', 'White')

print(colors[1:4]) # Slice from index 1 to 3
print(colors[:3]) # First 3 elements
```

OUTPUT

```
('Green', 'Blue', 'Yellow')
('Red', 'Green', 'Blue')
```

3.4 Tuple Immutability — Why It Matters

If you try to change the value of a tuple element, Python will raise a `TypeError`:

PYTHON CODE

```
colors = ('Red', 'Green', 'Blue')

colors[0] = 'Orange' # This will cause an error!
```

OUTPUT (ERROR)

```
TypeError: 'tuple' object does not support item assignment
```

When to Use a Tuple Instead of a List

Use a tuple when the data should remain constant (e.g., days of the week, month names). Tuples are faster than lists because Python can optimize them due to their immutability. Use tuples to represent fixed records — for example, (employee_id, name, department).

3.5 Tuple Length and Membership

PYTHON CODE

```
months = ('Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
          'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec')

print(len(months))    # Total count
print('Apr' in months) # Membership check
print('Xyz' in months)
```

OUTPUT

```
12
True
False
```

3.6 Creating a Single-Item Tuple

To create a tuple with just one item, you must add a **trailing comma**. Without the comma, Python treats it as a regular value in parentheses, not a tuple:

PYTHON CODE

```
not_a_tuple = ('Python') # This is just a string
yes_a_tuple = ('Python',) # This is a tuple

print(type(not_a_tuple))
print(type(yes_a_tuple))
```

OUTPUT

```
<class 'str'>
<class 'tuple'>
```

3.7 Common Tuple Methods

Method	Description	Example
count(x)	Returns how many times x appears	t.count(5)
index(x)	Returns the first index of x	t.index('Blue')

Tuples have only these two methods because they are immutable — there is no need for add, remove, or sort methods.

4 Complex Tuple Handling

4.1 Tuple Unpacking

Tuple unpacking allows you to assign the values inside a tuple to multiple variables in one single line. This is a very common and convenient technique in Python.

PYTHON CODE

```
student = ('Alice', 21, 'Computer Science')

name, age, department = student

print('Name:', name)
print('Age:', age)
print('Department:', department)
```

OUTPUT

```
Name: Alice
Age: 21
Department: Computer Science
```

Unpacking with * (Star Operator)

You can use `*` to capture multiple values into a list during unpacking:

PYTHON CODE

```
numbers = (1, 2, 3, 4, 5)

first, *middle, last = numbers

print('First:', first)
print('Middle:', middle)
print('Last:', last)
```

OUTPUT

```
First: 1
Middle: [2, 3, 4]
Last: 5
```

4.2 Nested Tuples

A tuple can contain other tuples as elements, just like nested lists. This is useful for representing structured data like employee records:

PYTHON CODE

```
employees = (  
    ('E001', 'Alice', 'Engineering'),  
    ('E002', 'Bob', 'Marketing'),  
    ('E003', 'Carol', 'Finance')  
)  
  
for emp in employees:  
    print(f'ID: {emp[0]}, Name: {emp[1]}, Dept: {emp[2]}')
```

OUTPUT

```
ID: E001, Name: Alice, Dept: Engineering  
ID: E002, Name: Bob, Dept: Marketing  
ID: E003, Name: Carol, Dept: Finance
```

4.3 Converting Between List and Tuple

Sometimes you need to convert a tuple to a list to modify it, and then convert it back. Use `list()` and `tuple()` for this:

PYTHON CODE

```
original_tuple = ('Python', 'Java', 'C++')  
  
# Convert to list to modify  
temp_list = list(original_tuple)  
temp_list.append('Rust')  
  
# Convert back to tuple  
updated_tuple = tuple(temp_list)  
print(updated_tuple)
```

OUTPUT

```
('Python', 'Java', 'C++', 'Rust')
```

4.4 Tuple as Function Return Values

Tuples are commonly used to return multiple values from a function. Python automatically packs multiple return values into a tuple:

PYTHON CODE

```
def get_dimensions():  
    width = 1920  
    height = 1080  
    return width, height # Returns as a tuple  
  
w, h = get_dimensions()  
print(f'Width: {w}, Height: {h}')
```

OUTPUT

```
Width: 1920, Height: 1080
```

4.5 List vs. Tuple — Side-by-Side Comparison

Feature	List	Tuple
Syntax	[] Square brackets	() Parentheses
Mutability	Mutable (can be changed)	Immutable (cannot be changed)
Speed	Slightly slower	Faster
Use Case	Dynamic collections	Fixed/constant data
Methods	Many (append, remove, sort...)	Only count() and index()
Memory	Uses more memory	Uses less memory

5 Basic Operations on Dictionary

5.1 What is a Dictionary?

A Dictionary in Python stores data as key-value pairs. Instead of accessing data by position (like lists and tuples), you access it by a unique key — similar to how a real dictionary works: you look up a word (key) to find its meaning (value).

Key Concept: Key-Value Pairs

Each entry in a dictionary has two parts: a key and a value.

Keys must be unique — no two entries can have the same key.

Keys must be of an immutable type: strings, numbers, or tuples.

Values can be of any type — numbers, strings, lists, or even other dictionaries.

Dictionaries are written using `{ }` (curly braces), with each key-value pair separated by a colon `:`.

Syntax

SYNTAX

```
dict_name = {  
    key1: value1,  
    key2: value2,  
    key3: value3  
}
```

Example 5.1 — Creating a Dictionary

PYTHON CODE

```
employee = {  
    'emp_id': 'E101',  
    'name': 'Alice',  
    'department': 'Engineering',  
    'salary': 75000  
}  
  
print(employee)
```

OUTPUT

```
{'emp_id': 'E101', 'name': 'Alice', 'department': 'Engineering', 'salary': 75000}
```

5.2 Accessing Values in a Dictionary

You can retrieve a value from a dictionary by using its key inside square brackets, or by using the `get()` method:

PYTHON CODE

```
employee = {'name': 'Alice', 'department': 'Engineering', 'salary': 75000}  
  
# Method 1: Direct key access  
print(employee['name'])  
  
# Method 2: Using get() — safer, returns None if key doesn't exist  
print(employee.get('salary'))  
print(employee.get('bonus')) # Key does not exist
```

OUTPUT

```
Alice  
75000  
None
```

Best Practice: Use get()

Using `employee['bonus']` when 'bonus' doesn't exist will raise a `KeyError` and crash your program.

Using `employee.get('bonus')` returns `None` safely — your program continues to run.

You can also provide a default value: `employee.get('bonus', 0)`

5.3 Adding and Updating Dictionary Entries

Adding a New Key-Value Pair

PYTHON CODE

```
employee = {'name': 'Alice', 'department': 'Engineering'}

employee['salary'] = 75000 # Adds a new key 'salary'
print(employee)
```

OUTPUT

```
{'name': 'Alice', 'department': 'Engineering', 'salary': 75000}
```

Updating an Existing Value

PYTHON CODE

```
employee = {'name': 'Alice', 'salary': 75000}

employee['salary'] = 82000 # Updates existing salary
print(employee)
```

OUTPUT

```
{'name': 'Alice', 'salary': 82000}
```

5.4 Removing Entries from a Dictionary

del — Remove by Key

PYTHON CODE

```
employee = {'name': 'Alice', 'department': 'Engineering', 'salary': 75000}

del employee['department']
print(employee)
```

OUTPUT

```
{'name': 'Alice', 'salary': 75000}
```

pop() — Remove and Return a Value

PYTHON CODE

```
employee = {'name': 'Alice', 'department': 'Engineering', 'salary': 75000}

removed = employee.pop('department')
print('Removed:', removed)
print('Remaining:', employee)
```

OUTPUT

```
Removed: Engineering
Remaining: {'name': 'Alice', 'salary': 75000}
```

5.5 Checking Keys and Length

PYTHON CODE

```
employee = {'name': 'Alice', 'department': 'Engineering', 'salary': 75000}

print(len(employee))      # Number of key-value pairs
print('name' in employee)  # Key exists?
print('bonus' in employee)
```

OUTPUT

```
3
True
False
```

5.6 Common Dictionary Methods — Quick Reference

Method	Description	Example
keys()	Returns all keys	d.keys()
values()	Returns all values	d.values()
items()	Returns all key-value pairs	d.items()
get(k, default)	Returns value for key k, or default	d.get('age', 0)
update(d2)	Merges d2 into the dictionary	d.update({'age': 25})
pop(k)	Removes and returns value for key k	d.pop('name')
clear()	Removes all entries	d.clear()
copy()	Returns a shallow copy	d2 = d.copy()

6 Complex Dict Handling

6.1 Iterating Through a Dictionary

You can loop through a dictionary to access its keys, values, or both.

Iterating Over Keys

PYTHON CODE

```
employee = {'name': 'Alice', 'department': 'Engineering', 'salary': 75000}

for key in employee:
    print(key)
```

OUTPUT

```
name
department
salary
```

Iterating Over Values

PYTHON CODE

```
for value in employee.values():
    print(value)
```

OUTPUT

```
Alice
Engineering
75000
```

Iterating Over Key-Value Pairs with items()

PYTHON CODE

```
for key, value in employee.items():
    print(f'{key}: {value}')
```

OUTPUT

```
name: Alice
department: Engineering
salary: 75000
```

6.2 Dictionary Comprehension

Similar to list comprehension, **dictionary comprehension** allows you to build a dictionary in a single concise line.

SYNTAX

```
{key_expression: value_expression for item in iterable}
```

Example — Squares Dictionary

PYTHON CODE

```
squares = {n: n**2 for n in range(1, 6)}  
print(squares)
```

OUTPUT

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Example — Filter with Condition

Create a new dictionary that includes only employees with salary above 60000:

PYTHON CODE

```
salaries = {'Alice': 75000, 'Bob': 52000, 'Carol': 90000, 'Dave': 48000}  
  
high_earners = {name: sal for name, sal in salaries.items() if sal > 60000}  
print(high_earners)
```

OUTPUT

```
{'Alice': 75000, 'Carol': 90000}
```

6.3 Nested Dictionaries

A dictionary can have other dictionaries as values. This is very useful for representing hierarchical or grouped data, such as a company's department structure.

PYTHON CODE

```
company = {  
    'Engineering': {  
        'head': 'Alice',  
        'team_size': 20  
    },  
    'Marketing': {  
        'head': 'Bob',  
        'team_size': 10  
    },  
    'Finance': {  
        'head': 'Carol',  
        'team_size': 8  
    }  
}
```

```
# Access nested values
print(company['Engineering']['head'])
print(company['Marketing']['team_size'])
```

OUTPUT

```
Alice
10
```

Iterating Through Nested Dictionary**PYTHON CODE**

```
for dept, info in company.items():
    print(f'Department: {dept}')
    print(f'  Head: {info["head"]}')
    print(f'  Team Size: {info["team_size"]}')

```

OUTPUT

```
Department: Engineering
  Head: Alice
  Team Size: 20
Department: Marketing
  Head: Bob
  Team Size: 10
Department: Finance
  Head: Carol
  Team Size: 8
```

6.4 Merging Dictionaries

Using update()**PYTHON CODE**

```
basic_info = {'name': 'Alice', 'age': 30}
job_info = {'department': 'Engineering', 'salary': 75000}

basic_info.update(job_info)
print(basic_info)
```

OUTPUT

```
{'name': 'Alice', 'age': 30, 'department': 'Engineering', 'salary': 75000}
```

Using the ** Unpacking Operator (Python 3.5+)

You can merge two dictionaries into a new one using ******:

PYTHON CODE

```
basic_info = {'name': 'Alice', 'age': 30}
job_info = {'department': 'Engineering', 'salary': 75000}

merged = {**basic_info, **job_info}
print(merged)
```

OUTPUT

```
{'name': 'Alice', 'age': 30, 'department': 'Engineering', 'salary': 75000}
```

6.5.setdefault() — Add a Key Only if It Does Not Exist

The `setdefault(key, default)` method is useful when you want to add a key-value pair to a dictionary only if the key does not already exist:

PYTHON CODE

```
employee = {'name': 'Alice', 'salary': 75000}

# 'department' does not exist, so it will be added
employee.setdefault('department', 'Unknown')
print(employee)

# 'salary' already exists, so it will NOT be changed
employee.setdefault('salary', 0)
print(employee)
```

OUTPUT

```
{'name': 'Alice', 'salary': 75000, 'department': 'Unknown'}
{'name': 'Alice', 'salary': 75000, 'department': 'Unknown'}
```

6.6 Practical Example — Student Grade Book

Let us build a small grade book system that uses a dictionary to store student names and their marks across multiple subjects:

PYTHON CODE

```
grade_book = {
    'Alice': {'Math': 88, 'Science': 92, 'English': 85},
    'Bob': {'Math': 75, 'Science': 80, 'English': 70},
    'Carol': {'Math': 95, 'Science': 88, 'English': 91}
}

# Print each student's average
for student, marks in grade_book.items():
```

```
avg = sum(marks.values()) / len(marks)
print(f'{student}: Average = {avg:.2f}')
```

OUTPUT

```
Alice: Average = 88.33
Bob: Average = 75.00
Carol: Average = 91.33
```

6.7 Summary — All Three Data Structures

Feature	List	Tuple	Dictionary
Syntax	[]	()	{ key: value }
Ordered	Yes	Yes	Yes (Python 3.7+)
Mutable	Yes	No	Yes
Indexed by	Position (0, 1, 2...)	Position (0, 1, 2...)	Keys (string, int, etc.)
Duplicates	Allowed	Allowed	Keys: No, Values: Yes
Best for	Dynamic ordered data	Fixed records	Key-value lookups

6.8 Chapter Summary

In this chapter, we covered all three major data structures in Python. Here is what we learned:

- **Lists** are ordered, mutable collections that support indexing, slicing, appending, removing, sorting, and list comprehension.
- **Tuples** are ordered, immutable sequences — ideal for storing fixed data. They support unpacking and can be nested.
- **Dictionaries** store data as key-value pairs, allowing fast lookup by key. They support nesting, comprehension, merging, and iteration over keys, values, and items.

Next Steps

Practice creating and manipulating lists, tuples, and dictionaries in the Python interactive shell.

Try to solve real-world problems — represent a product catalog as a dictionary, or a student roster as a list of tuples.

Experiment with nested structures — they are used extensively in JSON data, APIs, and databases.